

LP-II Laboratory Manual (Information Security)

Students have to choose any five Assignments from Following:

Write a Java/C/C++/Python program that contains a string (char pointer) with a value 'Hello World'. The program should AND or and XOR each character in this string with 127 and display the result.
Write a Java/C/C++/Python program to perform encryption and decryption using the method of Transposition technique.
Write a Java/C/C++/Python program to implement DES algorithm.
Write a Java/C/C++/Python program to implement AES Algorithm.
Write a Java/C/C++/Python program to implement RSA algorithm.
Implement the different Hellman Key Exchange mechanism using HTML and JavaScript. Consider the end user as one of the parties (Alice) and the JavaScript application as other party (bob).
Calculate the message digest of a text using the MD5 algorithm in JAVA.

Course: Information Security

Index

1. Program using AND and XOR with 127 on a string
 2. Transposition Cipher (Encryption & Decryption)
 3. DES Algorithm Implementation
 4. AES Algorithm Implementation
 5. RSA Algorithm Implementation
-

Prepared By :

**Dr. S. P. Pingat, Associate Professor,
Computer Engg. , SKNCOE**

Assignment No. 1: Bitwise AND and XOR on String Characters

Aim

To write a program that performs AND and XOR operations with 127 on each character of a given string.

Theory

At the lowest level, all data inside a computer system is stored and processed in binary form. Characters that appear as readable text to humans are internally represented using standardized encoding schemes such as ASCII or Unicode, where each character corresponds to a specific numerical value. These numerical values are stored as binary bit patterns. Understanding how these bits can be manipulated using logical operations is fundamental to understanding how encryption algorithms transform data to protect it from unauthorized access.

Bitwise operations are operations that directly manipulate individual bits of a binary number. The most commonly used bitwise operators in cryptographic transformations are AND, OR, XOR, NOT, and bit shifts. Among these, AND and XOR are especially important because they are computationally simple and form the basis of many encryption techniques. These operations are executed at the processor level and are extremely fast, which makes them suitable for high-speed encryption and decryption routines.

The AND operation compares corresponding bits of two numbers and produces a result where a bit is set to 1 only if both corresponding bits are 1. When a character is ANDed with the decimal value 127, which in binary is 01111111, the most significant bit (MSB) of the character is forced to 0 while the remaining bits remain unchanged. This technique is known as masking. Masking is widely used in networking, data encoding, and security applications to control or restrict certain bits of data. In this case, masking ensures that the character remains within the standard 7-bit ASCII range.

The XOR operation, also known as exclusive OR, produces a 1 when the two bits being compared are different and a 0 when they are the same. XOR has a special property that makes it extremely useful in cryptography: it is reversible. If a value A is XORed with B to produce C, then C XORed with B will return A. This reversibility is why XOR is commonly used in symmetric encryption algorithms, stream ciphers, and key mixing steps in block ciphers. It allows encryption and decryption to be performed using the same operation.

When AND and XOR operations are applied to each character in a string, the visible text gets transformed into a different sequence of characters. Although the transformation may not produce meaningful readable text, it clearly demonstrates how simple bitwise manipulations can obscure the original information. This illustrates the core principle of encryption: transforming data into a form that is not easily understood without applying the reverse transformation.

In modern cryptographic algorithms such as DES and AES, similar bitwise operations are applied repeatedly in complex sequences involving substitution boxes, permutation boxes, and key mixing steps. Therefore, this experiment serves as a foundation for understanding how large encryption algorithms operate internally at the bit level.

This experiment also reinforces understanding of character encoding, binary representation, logical operators, and reversible transformations, which are essential concepts for students learning cryptography and information security.

Algorithm

1. Start the program.
2. Define a string variable `s` and initialize it with the text **"Hello World"** (or any user input).
3. Display the original string.
4. Initialize two empty strings: `and_result` and `xor_result`.
5. Traverse the string character by character using a loop.
6. For each character `c` in the string:
 1. Convert the character into its ASCII integer value using `ord(c)`.
 2. Perform **bitwise AND** operation between the ASCII value and **127**.
 3. Convert the result back into a character using `chr()` and append it to `and_result`.
 4. Perform **bitwise XOR** operation between the ASCII value and **127**.
 5. Convert the result back into a character using `chr()` and append it to `xor_result`.
7. Continue the loop until all characters are processed.
8. Display the string obtained after the AND operation.
9. Display the string obtained after the XOR operation.

Stop the program

Pseudocode — Bitwise AND and XOR on String

```
BEGIN
    SET s ← "Hello World"
    DISPLAY "Original String: ", s

    SET and_result ← empty string
    SET xor_result ← empty string

    FOR each character c in s DO
        ascii ← ASCII value of c

        and_char ← CHARACTER of (ascii AND 127)
        APPEND and_char to and_result

        xor_char ← CHARACTER of (ascii XOR 127)
        APPEND xor_char to xor_result
    END FOR

    DISPLAY "String after AND with 127: ", and_result
    DISPLAY "String after XOR with 127: ", xor_result
END
```

Conclusion

This experiment demonstrated how characters stored in binary form can be manipulated using basic bitwise operations such as AND and XOR. By applying AND with 127, the most significant bit of each character was masked, ensuring the value remains within the 7-bit ASCII range. The XOR operation showed the important property of reversibility, which is widely used in encryption techniques. The practical execution of this program helped in understanding how simple bit manipulation can transform readable text into an altered form. This

experiment provided foundational knowledge required to understand how modern encryption algorithms internally operate at the bit level.

Sample Python Code

```
s = "Hello  
World"  
print("Original:", s)  
  
and_result = ''.join(chr(ord(c) & 127) for c in s)  
xor_result = ''.join(chr(ord(c) ^ 127) for c in s)  
  
print("AND with 127:", and_result)  
print("XOR with 127:", xor_result)
```

Output

Shows transformed strings after AND and XOR operations. Shows transformed strings after AND and XOR operations.

Assignment No. 2 : Transposition Cipher

Aim

To implement encryption and decryption using Transposition technique.

Theory

The Transposition Cipher is one of the earliest known encryption techniques used in classical cryptography. Unlike substitution ciphers where characters are replaced by other characters, transposition ciphers work by rearranging the positions of characters in the plaintext according to a systematic rule governed by a key. The actual letters remain unchanged, but their order is altered so that the message becomes unreadable without knowledge of the rearrangement pattern.

The basic principle behind transposition is permutation. A permutation is a rearrangement of elements in a particular order. In cryptography, permutation plays a crucial role in providing diffusion, which spreads the influence of each character over many positions in the ciphertext. This concept of diffusion is essential in modern encryption systems and was formally described by Claude Shannon as a requirement for secure cipher design.

One of the most popular forms of transposition cipher is the Columnar Transposition Cipher. In this method, the plaintext is written row-wise into a table with a fixed number of columns determined by a key. The ciphertext is generated by reading the table column-wise in an order determined by sorting the key. Because the key determines the reading order, the security of the cipher depends on keeping the key secret.

During decryption, the receiver reconstructs the table by filling in the columns according to the key order and then reads the rows to recover the original message. This process demonstrates that encryption and decryption are inverse operations based on the same permutation logic.

Historically, transposition ciphers were used in military and diplomatic communications because they were simple to apply manually but difficult to decipher without the key. Although modern computers can break simple transposition ciphers easily, the concept remains important because modern block ciphers like DES and AES use complex permutations as part of their internal rounds.

This experiment helps students understand indexing, data arrangement, matrix representation of text, and the concept of reversible transformations. It also introduces the idea that encryption does not always require changing characters; sometimes rearranging them is sufficient to hide meaning.

Studying transposition ciphers provides insight into how permutation contributes to encryption strength and prepares students for understanding permutation boxes used in advanced algorithms.

Algorithm (Columnar Transposition)

Encryption Algorithm

1. Start the program.
2. Accept the plaintext message from the user.
3. Accept the key (number of columns).
4. Create a list of empty strings equal to the key size to hold ciphertext columns.
5. For each column number from 0 to `key-1`:
 1. Set a pointer to the current column index.
 2. While the pointer is less than the message length:

- Append the character at the pointer position to the respective column string.
 - Increase the pointer by the key value.
6. Join all column strings to form the ciphertext.
 7. Display the encrypted message.

Decryption Algorithm

8. Accept the ciphertext and key.
9. Calculate the number of rows using message length and key.
10. Determine shaded boxes (unused cells in the grid).
11. Create a list of empty strings for plaintext rows.
12. Initialize row and column indices to 0.
13. For each character in ciphertext:
 - Append the character to the current row.
 - Move to the next row.
 - If the last row or shaded box condition is reached, reset row to 0 and move to next column.
14. Join the rows to obtain the original plaintext.
15. Display the decrypted message.
16. Stop the program.

Pseudocode — Transposition Cipher

Encryption

```
BEGIN
    INPUT message
    INPUT key

    CREATE array cipher[key] and initialize with empty strings

    FOR col ← 0 TO key-1 DO
        pointer ← col
        WHILE pointer < length of message DO
            cipher[col] ← cipher[col] + message[pointer]
            pointer ← pointer + key
        END WHILE
    END FOR

    ciphertext ← JOIN all elements of cipher
    DISPLAY "Encrypted Message: ", ciphertext
END
```

Decryption

```
BEGIN
    INPUT ciphertext
    INPUT key

    num_cols ← key
    num_rows ← CEILING(length of ciphertext / key)
    num_shaded ← (num_cols × num_rows) - length of ciphertext

    CREATE array plain[num_rows] with empty strings

    col ← 0
    row ← 0

    FOR each symbol in ciphertext DO
        plain[row] ← plain[row] + symbol
        row ← row + 1
    END FOR
```

```

IF row = num_rows OR
  (row = num_rows - 1 AND col ≥ num_cols - num_shaded) THEN
  row ← 0
  col ← col + 1
END IF

```

Conclusion — Transposition Cipher

This experiment successfully implemented encryption and decryption using the transposition cipher technique. It demonstrated how rearranging the positions of characters without changing them can make a message unreadable. The concept of permutation and its role in providing diffusion was clearly understood through this implementation. The reversible nature of the process highlighted how encryption and decryption are inverse operations. This experiment helped in understanding the importance of character arrangement and prepared the foundation for studying permutation techniques used in modern block ciphers.

Sample Python Code

```

def encrypt(message, key):
    cipher = [''] * key
    for col in range(key):
        pointer = col
        while pointer < len(message):
            cipher[col] += message[pointer]
            pointer += key
    return ''.join(cipher)
```python
def encrypt(message, key):
 cipher = [''] * key
 for col in range(key):
 pointer = col
 while pointer < len(message):
 cipher[col] += message[pointer]
 pointer += key
 return ''.join(cipher)

def decrypt(cipher, key):
 num_cols = key
 num_rows = len(cipher) // key + (len(cipher) % key != 0)
 num_shaded = (num_cols * num_rows) - len(cipher)
 plain = [''] * num_rows
 col = 0
 row = 0

 for symbol in cipher:
 plain[row] += symbol
 row += 1
 if (row == num_rows) or (row == num_rows - 1 and col >= num_cols - num_shaded):
 row = 0
 col += 1

 return ''.join(plain)

```

# Assignment No. 3: DES Algorithm

## Aim

To implement DES encryption and decryption.

## Theory

The Data Encryption Standard (DES) is a symmetric key block cipher that played a historic role in the development of modern cryptography. It operates on 64-bit blocks of data and uses a 56-bit effective key. DES was adopted as a federal standard and became the most widely used encryption technique for several decades.

DES is based on the Feistel network structure, which divides the data block into two halves and processes them through 16 rounds of identical operations. In each round, a round key derived from the main key is used along with substitution and permutation operations. The Feistel structure ensures that the same process can be reversed for decryption by applying the round keys in reverse order.

A major strength of DES lies in its use of S-boxes (substitution boxes) and P-boxes (permutation boxes). S-boxes introduce non-linearity into the encryption process, making it resistant to linear and differential cryptanalysis. P-boxes spread bits across positions to provide diffusion. Together, these create the properties of confusion and diffusion necessary for secure encryption.

DES also includes an initial permutation and a final permutation, which rearrange the bits before and after the rounds. Although these permutations do not add security, they were designed to optimize hardware implementation at the time DES was developed.

Despite its strong design, DES is now considered insecure due to its small key size, which makes it vulnerable to brute-force attacks. However, it remains extremely important for educational purposes because it clearly demonstrates the structure of a block cipher and the principles used in modern algorithms.

Understanding DES helps students learn about key scheduling, round functions, substitution, permutation, and the concept of iterative encryption rounds that strengthen security.

## Algorithm — DES Encryption and Decryption

1. Start the program.
2. Import DES cipher and padding utilities.
3. Define an 8-byte secret key.
4. Create a DES cipher object in ECB mode using the key.
5. Accept plaintext input from the user in byte form.
6. Apply padding to the plaintext to make its length a multiple of 8 bytes.
7. Encrypt the padded plaintext using the DES cipher.
8. Display the encrypted ciphertext.
9. Decrypt the ciphertext using the same DES cipher.
10. Remove padding from the decrypted data.
11. Display the original decrypted plaintext.

Stop the program.

## Pseudocode — DES Encryption and Decryption



```

BEGIN
 IMPORT DES library

 SET key ← 8-byte secret key
 CREATE DES cipher using key in ECB mode

 INPUT plaintext
 padded_text ← PAD plaintext to multiple of 8 bytes

 encrypted_text ← ENCRYPT padded_text using DES
 DISPLAY "Encrypted Text: ", encrypted_text

 decrypted_padded ← DECRYPT encrypted_text using DES
 decrypted_text ← REMOVE padding

 DISPLAY "Decrypted Text: ", decrypted_text
END

```

## Conclusion — DES Algorithm

This experiment demonstrated the working of the DES symmetric key encryption algorithm using a programming library. The process of padding, encryption, and decryption was successfully performed. Through this implementation, the concepts of block cipher, Feistel structure, and the use of a secret key for both encryption and decryption were understood. Although DES is now outdated due to its small key size, studying it provided valuable insight into the structure and design principles of modern encryption algorithms.

### Sample Python Code (using library)

```

from Crypto.Cipher import DES
```python
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = b'8bytekey'
cipher = DES.new(key, DES.MODE_ECB)

text = b'HelloDES'
encrypted = cipher.encrypt(pad(text, 8))
decrypted = unpad(cipher.decrypt(encrypted), 8)

print(encrypted)
print(decrypted)

```

Assignment No. 4: AES Algorithm

Aim

To implement AES encryption and decryption.

Theory

The Advanced Encryption Standard (AES) is the current global standard for symmetric key encryption. It was developed to replace DES and provides much stronger security with efficient performance. AES operates on 128-bit blocks and supports key sizes of 128, 192, and 256 bits.

AES is based on a substitution–permutation network rather than a Feistel structure. It performs multiple rounds (10, 12, or 14 depending on key size), and each round includes four operations: SubBytes, ShiftRows, MixColumns, and AddRoundKey. These operations combine substitution, permutation, matrix multiplication, and key mixing to achieve high levels of confusion and diffusion.

SubBytes uses a carefully designed S-box derived from finite field mathematics to replace each byte. ShiftRows rearranges bytes to introduce permutation. MixColumns performs matrix multiplication in a finite field to spread information across bytes. AddRoundKey combines the state with a portion of the key using XOR.

AES is highly secure against known attacks and is widely used in secure communication protocols, file encryption, disk encryption, and wireless security. Its design allows fast implementation in both hardware and software environments.

Studying AES provides students with knowledge of modern cipher design, finite field arithmetic, and how structured rounds create strong encryption. It highlights why AES is trusted worldwide for protecting sensitive data.

Algorithm — AES Encryption and Decryption

1. Start the program.
2. Import AES cipher and padding utilities.
3. Define a 16-byte secret key.
4. Create an AES cipher object in ECB mode using the key.
5. Accept plaintext input from the user in byte form.
6. Apply padding to ensure the plaintext is a multiple of 16 bytes.
7. Encrypt the padded plaintext using AES.
8. Display the encrypted ciphertext.
9. Decrypt the ciphertext using the same AES cipher.
10. Remove padding from decrypted data.
11. Display the original plaintext.

Stop the program.

Pseudocode — AES Encryption and Decryption

```
BEGIN
    IMPORT AES library

    SET key ← 16-byte secret key
```

```
CREATE AES cipher using key in ECB mode

INPUT plaintext
padded_text ← PAD plaintext to multiple of 16 bytes

encrypted_text ← ENCRYPT padded_text using AES
DISPLAY "Encrypted Text: ", encrypted_text

decrypted_padded ← DECRYPT encrypted_text using AES
decrypted_text ← REMOVE padding

DISPLAY "Decrypted Text: ", decrypted_text

END
```

Conclusion — AES Algorithm

This experiment successfully implemented AES encryption and decryption. It demonstrated how modern symmetric encryption algorithms operate on fixed-size blocks using substitution and permutation operations along with key mixing. The process of padding, encryption, and recovery of original data validated the correctness of the algorithm. This experiment helped in understanding why AES is considered secure and widely adopted for protecting sensitive information in real-world applications.

Sample Python Code

```
from Crypto.Cipher import AES
```python
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

key = b'0123456789abcdef'
cipher = AES.new(key, AES.MODE_ECB)

text = b'HelloAESWorld!!'
encrypted = cipher.encrypt(pad(text, 16))
decrypted = unpad(cipher.decrypt(encrypted), 16)

print(encrypted)
print(decrypted)
```

---

# Assignment No. 5: RSA Algorithm

## Aim

To implement RSA encryption and decryption.

## Theory

RSA is a public key cryptographic algorithm that enables secure communication without sharing secret keys beforehand. It uses a pair of keys: a public key for encryption and a private key for decryption. This asymmetric nature makes RSA extremely useful for secure key exchange and digital signatures.

The security of RSA is based on the mathematical difficulty of factoring large composite numbers into their prime factors. During key generation, two large prime numbers are selected and multiplied to produce a modulus. The public and private keys are derived using Euler's totient function and modular arithmetic.

Encryption is performed using the recipient's public key, and decryption is done using the private key. Because the private key is never shared, RSA ensures confidentiality and authentication.

RSA is widely used in SSL/TLS certificates, email encryption, and digital signatures. Although slower than symmetric algorithms, it is ideal for securely exchanging symmetric keys.

This experiment introduces students to number theory, modular exponentiation, and the mathematics that underpin modern cryptography.

## Algorithm — RSA Encryption and Decryption

1. Start the program.
2. Import RSA key generation and encryption libraries.
3. Generate a 2048-bit RSA key pair.
4. Extract the public key from the generated key.
5. Accept plaintext message from the user in byte form.
6. Create an encryption object using the public key and OAEP padding.
7. Encrypt the plaintext using the public key.
8. Display the encrypted ciphertext.
9. Create a decryption object using the private key.
10. Decrypt the ciphertext using the private key.
11. Display the original decrypted plaintext.
12. Stop the program.

## Pseudocode — AES Encryption and Decryption

```
BEGIN
 IMPORT AES library

 SET key ← 16-byte secret key
 CREATE AES cipher using key in ECB mode
```

```
INPUT plaintext
padded_text ← PAD plaintext to multiple of 16 bytes

encrypted_text ← ENCRYPT padded_text using AES
DISPLAY "Encrypted Text: ", encrypted_text

decrypted_padded ← DECRYPT encrypted_text using AES
decrypted_text ← REMOVE padding

DISPLAY "Decrypted Text: ", decrypted_text

END
```

## Conclusion — RSA Algorithm

This experiment demonstrated the implementation of RSA, an asymmetric key encryption algorithm. The generation of public and private keys, encryption using the public key, and decryption using the private key were successfully performed. This experiment provided a clear understanding of how secure communication is possible without sharing secret keys in advance. It also introduced the mathematical concepts behind RSA and highlighted its importance in secure key exchange and digital signatures.

### Sample Python Code

```
from Crypto.PublicKey import RSA
```python
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

key = RSA.generate(2048)
public_key = key.publickey()

cipher = PKCS1_OAEP.new(public_key)
message = b'HelloRSA'
encrypted = cipher.encrypt(message)

cipher = PKCS1_OAEP.new(key)
decrypted = cipher.decrypt(encrypted)

print(decrypted)
```
